Práctica 1: PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Gabriel Camilo Quijano Celis - 2220387
Andres Felipe Velasquez Gomez - 2224561
Jeferson Uribe Correa - 2220428

https://github.com/JefersonUIS/2026_1_CommII_A1_G9

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

Febrero 28, 2026

***Abstract*—This work implements Software Defined Radio (SDR) concepts using GNU Radio and custom Python blocks for digital signal processing. Algorithms for accumulation, differentiation, and real-time statistical analysis were developed and tested on discrete-time signals. The system's performance was evaluated under variable sampling rates and noisy environments, validating theoretical digital communication models. Results demonstrate the efficiency of programmable blocks for signal characterization and robust experimentation in SDR platforms.**

Keywords: *SDR, GNU Radio, Python, Signal Processing, Accumulator, Differentiator.*

I INTRODUCCIÓN

La Radio Definida por Software (SDR) es fundamental en las telecomunicaciones modernas por su capacidad de implementar esquemas de procesamiento mediante software [1]. Plataformas como GNU Radio permiten diseñar y evaluar sistemas digitales utilizando diagramas de flujo que representan etapas críticas del tratamiento de señales [2].

Este trabajo presenta el desarrollo de bloques personalizados en Python para ampliar las capacidades nativas de GNU Radio. Se implementaron algoritmos de acumulación y diferenciación para analizar el comportamiento de señales en tiempo real mediante herramientas de visualización. Asimismo, se integraron operadores de análisis estadístico para el cálculo de parámetros como la media, valor RMS, potencia promedio y desviación estándar [3].

Estas implementaciones fortalecen la comprensión de conceptos teóricos aplicados en entornos prácticos, evidenciando la relevancia de la SDR como plataforma experimental para la validación de sistemas de comunicación.

II METODOLOGÍA

El desarrollo de la práctica se dividió en tres fases principales: configuración del entorno, diseño de bloques y validación estadística.

II-A Diseño e Implementación de Bloques

Se utilizó GNU Radio como entorno de procesamiento, empleando *Embedded Python Blocks* para programar algoritmos de manipulación de señales discretas:

- **Acumulador (Integrador):** Calcula la suma acumulada de las muestras de entrada para procesos de recuperación de señales:

$$y[n] = \sum_{k=0}^{n} x[k] \tag{1}$$

- **Diferenciador:** Resalta las variaciones rápidas en la amplitud mediante la diferencia entre muestras consecutivas:

$$y[n] = x[n] - x[n-1] \tag{2}$$

- **Operadores Estadísticos:** Se desarrollaron algoritmos para la extracción de métricas en tiempo real, incluyendo media ($\bar{x}$), valor eficaz (RMS), desviación (σ_x) y potencia (P) [3].

II-B Validación y Robustez

Se validó la precisión de los bloques inyectando ruido blanco gaussiano (0 a 0.1) a una señal senoidal pura. Esta fase permitió contrastar los cálculos teóricos frente a la ejecución práctica, confirmando la fiabilidad de los algoritmos ante señales estocásticas.

III RESULTADOS

Se analizó el comportamiento de los bloques en GNU Radio mediante algoritmos de acumulación, diferenciación y análisis estadístico.

III-A Acumulador

El acumulador actúa como un sistema integrador que suaviza variaciones rápidas al compensar componentes de ruido mediante la suma de muestras. Para su evaluación, se configuró una frecuencia de 32 kHz con un *Vector Source* de 8 muestras y un *Throttle* Véase la Fig. 1.

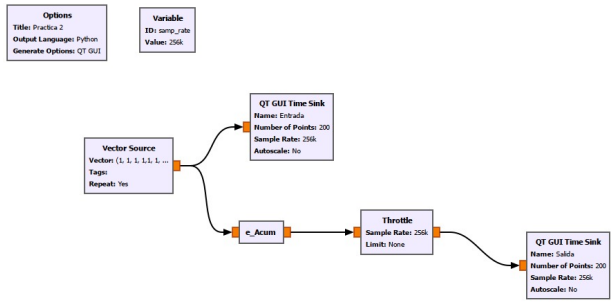

Figure 1. Diagrama de bloques del acumulador.

Análisis de Señal: Ante una señal cuadrada $\pm A$, el bloque generó como salida una señal de rampa (Fig. 2), confirmando el efecto integrador en tiempo discreto.

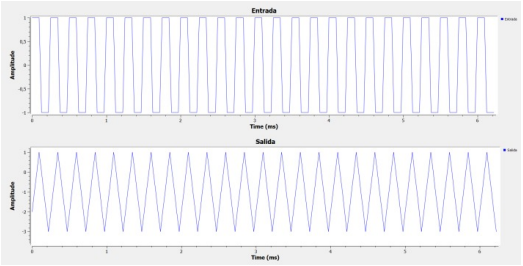

Figure 2. Señal de entrada y salida (parámetros iniciales).

Efecto de Memoria y Muestreo: Al variar el número de muestras en el bloque, la salida conserva estados previos generando variaciones acumulativas y un desplazamiento en la amplitud (Fig. 3). Este fenómeno ratifica la naturaleza recursiva del acumulador, donde la persistencia del estado interno permite la integración de la señal a lo largo del tiempo.

Por otro lado, se observó que si bien aumentar la frecuencia de muestreo mejora la continuidad de la señal, una reducción drástica incrementa significativamente el retardo de fase del sistema (Fig. 4). Esto demuestra que la tasa de muestreo no solo limita la resolución temporal, sino que condiciona directamente la velocidad de respuesta y la estabilidad del algoritmo empleado para esta situación experimental.

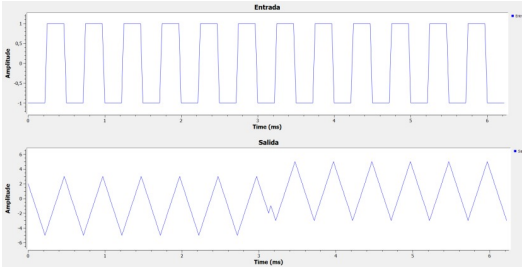

Figure 3. Respuesta con variación en el número de muestras.

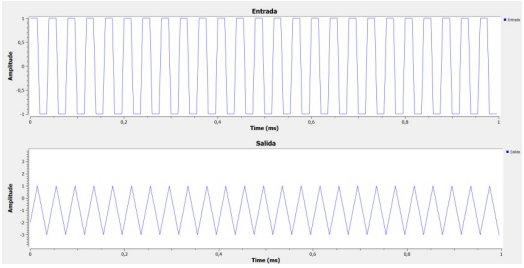

Figure 4. Respuesta con variación en la frecuencia de muestreo.

III-B Bloque Diferenciador

El diferenciador es un sistema que procesa una señal de entrada para resaltar los cambios rápidos o transiciones en su amplitud.

Diagrama de Flujo del bloque "Diferenciador"

Se hizo uso del bloque diferenciador empleando una frecuencia de muestreo de 32 kHz. Se emplearon tres señales de entrada generadas mediante el bloque Vector Source, integrando un Throttle para regular la velocidad de procesamiento. En la Fig. 5 se muestra el diagrama correspondiente a esta implementación.

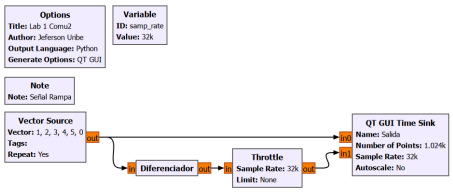

Figure 5. Diagrama de flujo y respuesta del bloque diferenciador ante una entrada tipo rampa.

Señal Constante Para esta prueba, se usó una señal constante de valor 5.

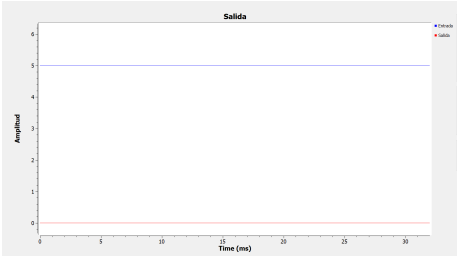

Figure 6. Gráfica de la señal de entrada y salida del bloque diferenciador con señal constante como entrada.

En la Fig. 6 obtenida, la señal de salida tomó únicamente valores de 0, esto se da porque, al no haber ninguna variación en la amplitud de la entrada, el bloque realiza constantemente la resta de la muestra actual menos la anterior (5 - 5). Esto confirma que el diferenciador solo toma valores cuando observa cambios en la señal de entrada.

Señal de Rampa Se tomó una señal de rampa como señal de entrada con magnitud máxima de 5 y con pendiente 1.

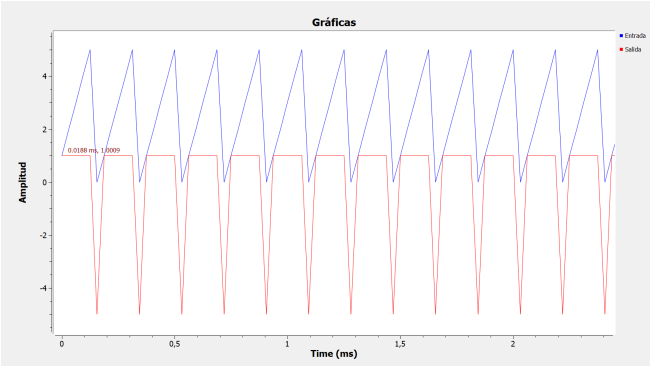

Figure 7. Gráfica de la señal de entrada y salida del bloque diferenciador con señal rampa como entrada.

En la gráfica obtenida Fig. 7, se observa que mientras la rampa de entrada crece de manera lineal, la señal de salida se mantiene en un valor constante de 1. Esto es porque la señal de entrada tiene una pendiente uniforme y de valor 1; es decir, el cambio de valor entre una muestra y la siguiente será siempre de 1. Por otro lado, en el momento en que la rampa alcanza su valor máximo, en este caso 5, y luego cae a 0, en este instante, el bloque diferenciador realiza la operación de restar el valor actual (0) menos el anterior (5), lo que genera ese pico de magnitud -5 que se logra apreciar en la gráfica.

Señal Cuadrada Y para finalizar con los ensayos del bloque diferenciador, la señal constante fue cambiada por una señal cuadrada de magnitud 1.

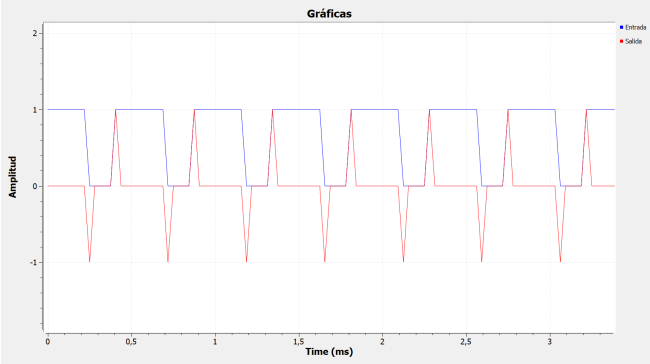

Figure 8. Gráfica de la señal de entrada y salida del bloque diferenciador con señal cuadrada como entrada

Pudimos notar en la Fig. 8, que en la salida aparecieron dos picos muy claros: uno positivo al principio y otro negativo al final del pulso. El primer pico ocurre porque, cuando la señal arranca, pasa de 0 a 1 de golpe, y como nuestro código resta el valor actual menos el anterior, el resultado es 1 positivo. El segundo pico, toma valores negativos ya que se da cuando el pulso termina y la señal vuelve a cero,ahí la operación es al revés (0 - 1), lo que nos da el -1 que se ve hacia abajo. Básicamente, estos picos nos sirven para identificar exactamente los bordes o "flancos" de la señal, avisándonos justo cuando empieza y cuando termina cada pulso azul, o sea de la señal de entrada.

III-C Análisis Estadístico en Tiempo Real

Se implementó un diagrama de flujo (Fig. 9) para calcular parámetros métricos de diversos vectores de entrada. Inicialmente, se verificó la precisión de los bloques inyectando ruido blanco gaussiano a una señal senoidal, contrastando los valores teóricos frente a la ejecución práctica (Tabla 1).

Table 1
VALIDACIÓN DE BLOQUES: TEÓRICO (1) VS. EJECUCIÓN (2) BAJO RUIDO.

Ruido	Media	RMS	Potencia	Desv. Est.
0 (1)	0	0.7073	0.5002	0.7073
0 (2)	0	0.7071	0.5000	0.7071
0.1 (1)	0	0.7152	0.5112	0.7141
0.1 (2)	0.0002	0.7138	0.5100	0.7137

A partir de esta validación, se generaron diferentes señales para calcular los parámetros de cada vector, cuyos resultados se sintetizan en la Tabla 2.

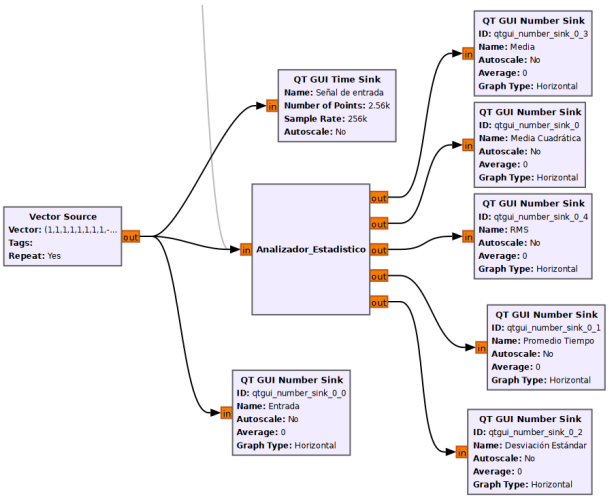

Figure 9. Diagrama de flujo para análisis estadístico.

Los datos obtenidos demuestran que la frecuencia de muestreo no alteró los valores estadísticos (RMS, Media y Desviación constantes), confirmando que las métricas dependen estrictamente de los valores del vector y no de la velocidad de procesamiento.

Por el contrario, el número de muestras afecta la precisión de la media (variación a 0.73m con 16 muestras), mientras que la alteración del vector incrementa drásticamente el RMS (valores de 4.43 y 3.57), validando la sensibilidad de los bloques ante cambios en la energía de la señal.

Table 2
RESULTADOS DE BLOQUES ESTADÍSTICOS CON VARIACIONES DE F_s Y N.

Nº Muest.	Fs [kHz]	RMS	Media	Desv.	Prom.	M. Cuad.
8	32	1	0	1	1	1
16	32	1	0.73m	1	1	1
8	2	1	0	1	1	1
8	320	1	0	1	1	1
8	32	4.43	0.125	4.44	19.62	19.62

III-D Aplicación en Sistemas de Audio

El análisis estadístico es vital para el control de señales sonoras. La Media detecta desplazamientos DC indeseados; el RMS mide la sonoridad percibida (loudness); la Desviación Estándar refleja la dinámica para el diseño de compresores; y la Media Cuadrática estima la energía base en sistemas de potencia. Estas métricas permiten optimizar procesos de mezcla, ecualización y transmisión digital.

IV CONCLUSIONES

- Se comprobó que el bloque acumulador actúa como un integrador en tiempo discreto al transformar señales de entrada cuadradas en señales de salida tipo rampa, evidenciando una naturaleza recursiva donde la persistencia del estado interno permite la integración de la señal a lo largo del tiempo.
- Se verificó que el bloque diferenciador identifica con precisión los flancos o transiciones de las señales digitales mediante la generación de picos de magnitud proporcional al cambio de nivel, demostrando su capacidad para transformar rampas en niveles de voltaje constantes.
- El análisis de la tasa de muestreo evidenció que una reducción drástica de frecuencia incrementa significativamente el retardo de fase del sistema, sin alterar valores estadísticos como el RMS o la media, lo que confirma que estas métricas dependen de los valores del vector y no de la velocidad de procesamiento.
- Se determinó que la precisión de la media es sensible al número de muestras procesadas, mientras que los cambios en la energía de la señal se reflejan drásticamente en el valor RMS, validando la sensibilidad del bloque para la caracterización de señales estocásticas.
- La implementación de métricas en tiempo real resultó fundamental para aplicaciones de audio, permitiendo la detección de desplazamientos DC mediante la media y la medición de sonoridad percibida (*loudness*) a través del RMS, parámetros críticos para la optimización de mezcla y diseño de compresores.

References

[1] V. K. Garg, *Comunicaciones y redes inalámbricas*. Morgan Kaufmann, 2007.
[2] GNU Radio Project, "GNU Radio Documentation." [Online]. Available: https://www.gnuradio.org
[3] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2010.